CSE 2421

headers, storage, scope

Scope

- Scope
  - *A region of a program in which a variable can be accessed*
  - No one can access the variable outside of its scope
  - It is legal to declare different variables of the same name so long as they are not declared in the same scope
- *The location where a variable is declared determines its scope*

Block Scope

```
if (...) {
    /* outer block */
    int x;
    if (...) {
        /* inner block */
        int y;
    }
}
```

- A variable declared in a block can be accessed by any statement within that block, or a nested (inner) block
  - x is accessible to both the inner and outer block
  - y is accessible to only the inner block.

Block Scope Examples

```
/* block scope */

void f()
{
    /* accessible only inside f() */
    int x=0;

    if (x) {
        /* accessible only inside this if */
        int y=0;
    }

    /* invalid!  outside of y's scope */
    x=y;
}
```

Block Scope

- C89
  - All variables must be declared at the beginning of a block
- C99
  - Variables may be declared anywhere in the block

Block Scope Examples

```c
/* C89 version */

void f()
{
    int i;
    char c;

    for (i=0;i<10;i++) {
        ...
    }

    c=getchar();
}
```

```c
/* C99 version */

void f()
{
    for (int i=0;i<10;i++) {
        ...
    }

    char c;

    c=getchar();
}
```

File Scope (i.e. Global)

- Any identifier declared outside of all blocks
- The variable may be accessed anywhere from its declaration to the end of the source file in which it was declared

```
int f() {
    /* cannot access x */
}

int x=0; /* file scope variable */

void g() {
    /* can access x */
}
```

- Usually, file scope variables are declared before functions, to avoid this problem

Prototype Scope

- Applies to variable names given in a function prototype
- Remember it is not required to supply names
- Prototype scope extends uniquely to each prototype

```
//Valid prototypes
void myFunction1(int var1, int var2);
void myFunction2(int var1, int var2);
```

Scope Example

```
int a;
int b (int c);
int d (int e)
{
   int f;
   …
   {
      int g, h;
      …
   }
   {
       int h;
      …
   }
}
```

```
1 → int a;
2 → int b          3 → int c
4 → int d          5 → int e

6 → int f

7 → int g, h


8 → int h
```

File scope = ?
Prototype scope = ?
Block = ?

Scope Example

```
int a;
int b (int c);
int d (int e)
{
   int f;
   …
   {
      int g, h;
      …
   }
   {
       int h;
      …
   }
}
```

```
1 → int a;
2 → int b          3 → int c
4 → int d          5 → int e

6 → int f

7 → int g, h


8 → int h
```

File scope = 1,2, 4
Prototype scope = 3
Block = 5,6, 7, 8

Symbol Table

- While compiling: the compiler takes every function of variable declaration and adds it to a symbol table
  - `<Symbol> <Address>`
- While compiling: when encountering a symbol being used the compiler looks at its table to find if its valid
- *Understanding* how the compiler is building the symbol table is easier than memorizing all of the scope / linkage rules
- Compiler typically takes a top down approach

Function Calling

- C is unable to compile a call that is not in its symbol table.

```
int main() {
    myfunc();   //Invalid!
}
void myfunc() {
..
}
```

How does C become aware of functions?

- functions that physically appear earlier in the file
  - By declaring the function earlier in the file, by the time the compiler reaches the function call, the function will have already been added to the symbol table

```
void myfunc() {
    ..
}
int main() {
    myfunc();   //Valid!
}
```

How does C become aware of functions?

- Function prototypes
- Results in a partial symbol table entry. The prototype allows the symbol to be added to the table,
- The address is not known until the function definition is reached

```
void myfunc(); //prototype

int main() {
    myfunc();  //Valid!
}
void myfunc() {
    ..
}
```

Function Prototypes

- Matching definition for the function but without any implementation.
  - Variable types are required, Variable names are not
    - If variable names are supplied they need not match the function names

Function Prototypes

```
void myFunction(int a, long b) {
…..
}
```

Which are valid prototypes?

```
void myFunction(int a, long b);
void myFunction(int b, long a);
void myFunction(a, b);
void myFunction(int, long);
```

Function Prototypes

```
void myFunction(int a, long b) {
…..
}
```

Which are valid prototypes?

```
void myFunction(int a, long b);
void myFunction(int b, long a); //tricky, but valid
void myFunction(a, b); //Types are required
void myFunction(int, long);
```

How does C become aware of functions?

- Header files (.h) are a method for sharing prototypes between .c files

```
//fileA.h

function1(int a, int b);
function2(long a, long b);
function3(int, long);
```

```
//fileB.c
#include <fileA.h>

int main() {
	int x = 0;
	long y = 1L;
	function3(x,y);
}
```

Function Accessibility

- Java provides accessibility keywords
  - public
  - private
  - protected
- C does not offer any such keywords
  - Similar functionality can be achieved by coding smart

Header Files

- Header files are how you share functions
  - Only put what you want to be ‘public’ in them

```
//fileA.h
function1(int a, int b);
function2(long a, long b);


//fileA.c

function3(int, long);
function1(int a, int b) {
..}
function2(long a, long b) {
… }
function3(int, long) {
… }
```

```
//fileB.c
#include “fileA.h”

int main() {
    int x = 0;
    long y = 1L;
    function3(x,y); //INVALID
    function1(x,x);
}
```

Function Namespace?

- All functions by default share the same namespace
  - global namespace

```
//a.h
myFunction(int a);
```

```
//b.h
myFunction(int a);
```

```
//main.c
#include “a.h”
#include “b.h”

int main() {
	int x = 0;
	myFunction(x);
}
```

gcc: undefined reference to `myFunction'
collect2: error: ld returned 1 exit status

Function Namespace?

- Without separate namespaces there is no way to specify myFunction from A.h or myFunction from b.h
- Compiler sees the first myFunction definition and adds it to the symbol table
- Compiler hits the second one, sees it has already been declared and throws an error

```
//b.h
myFunction(int a);
```

```
//A.h
myFunction(int a);
```

Function Namespace?

- All functions by default share the same namespace
  - global namespace
  - C doesn't support overloading

```
//A.h
myFunction(int a);
```

```
//b.h
myFunction(int a, long x);
```

```
//main.c
#include "a.h"
#include "b.h"
int main() {
    int x = 0;
     myFunction(x);
}
```

```
gcc:
In file included from main.c:2:0:
b.h:1:6: error: conflicting types for 'myFunction'
In file included from main.c:1:0:
a.h:1:6: note: previous declaration of 'myFunction' was here
```

Namespace?

- Function names must be unique
- Not just methods shared in header files
  - Methods kept ‘private’ are still global in namespace at compile time

```
//fileA.h
function1(int a, int b);

//fileA.c
function3(int, long);
function1(int a, int b) {
..}
function3(int, long) {
… }
```

```
//fileB.c
#include “fileA.h”
function3(int, long);
int main() {
    int x = 0;
    long y = 1L;
    function3(x,y); //INVALID
    function1(x,x);
}
void function3(int a, long x) {
… }
```

Variable Namespace

- Variable sharing follows the same rules as Functions

```
//fileA.h
int x = 10;
```

```
//fileA.c

int y = 0;
```

```
//fileB.c
#include "fileA.h"

int main() {
    if(x > 5){      //VALID
        …
    }
    if(y > 5) { //INVALID
    ….
    }
}
```

Variable Namespace

- Variable sharing follows the same rules as Functions

```
//fileA.h
int x;
```

```
//fileA.c

void init() {
   x = 10;
}
```

```
//fileB.c
#include "fileA.h"

int main() {
   if(x > 5){//Invalid
      …
   }
```

Not actually allocating space for x! Just declaring it... How would we fix?

Defining vs. Declaring Variable

- A variable is **defined** when the compiler **allocates the storage** for the variable.
  - int x = 10;
- A variable is **declared** when the compiler is informed that a variable exists (and this is its type); it **does not allocate** the storage for the variable at that point.
  - int x;

Extern Keyword

- Allows a variable *declaration* to exist in a header file
  - Only one header file may declare the variable
- One source file that includes the header must *define* the variable.
  - Only one source file may define the variable

Variable Namespace

- Variable sharing follows the same rules as Functions

```
//fileA.h
extern int x;
```

```
//fileA.c
int x = 10;
void init() {
}
```

```
//fileB.c
#include "fileA.h"

int main() {
   if(x > 5){//VALID
      …
   }
```

Variable Namespace

- Variable sharing follows the same rules as Functions

```
//fileA.h
extern int x;
```

```
//fileA.c
int x = 10;
void init() {
}
```

```
//fileB.c
#include "fileA.h"
int x = 30; //INVALID!
int main() {
   if(x > 5){
      …
   }
```

- Can't have two defines for x

Variable Accessibility

```
int main() {
    temp++;   //Invalid!
}

int temp = 9;
```

- When the compiler reaches ‘temp++’ there is no temp variable in the compiler's symbol table

Variable Accessibility

```
int temp = 9;

int main() {
    temp++;   //Valid!
}
```

By declaring it at the beginning of the file temp will exist in the symbol table before it's used

Two methods of the same name?

- Having two methods with the same name in your project causes a compile time conflict
- How can we get around this?
- Static keyword
  - When used on methods it makes the method internal to that file at build time

Static Methods

```
//fileA.h
void function1(int a, int b); // externally visible
```

```
// fileA.c
#include <stdio.h>
static function3(int, long);
void function1(int a, int b){

	function3(a, b);
}

static function3(int a, long b)
{
	printf("function3 in fileA\n");
}
```

```
//fileB.c
#include <stdio.h>
#include "fileA.h"

static function3(int, long);
int main()
{
	int x = 0;
	long y = 1L;
	function3(x, y);
	function1(x, x);
}
static function3(int a, long x)
{
	printf("function3 in fileB\n");
}
```

gcc fileA.c fileB.c && ./a.out

Static Method Example

- Why might you need two methods of the same name?
- Common Utility methods

```
//Person.h

static void init();
static void print_info();
```

```
//Animal.h

static void init();
static void print_info();
```

Static Method Example (2)

```c
//helper_file.c

int f1(int);            /* prototype */
static int f2(int);     /* prototype */

int f1(int foo) {
   return f2(foo);      /* ok, f2 is in the same translation unit */
                        /* (basically same .c file) as f1         */
}

int f2(int foo) {
   return 42 + foo;
}
```

```c
//main.c:

int f1(int); /* prototype */
int f2(int); /* prototype */

int main(void) {
   f1(10); /* ok, f1 is visible to the linker */
   f2(12); /* nope, f2 is not visible to the linker */
   return 0;
}
```

http://stackoverflow.com/questions/5319361/static-function-in-c

CSE 2421

Storage Class

Storage Class

- Refers to the type of memory in which the variable's value is stored which in turn defines different characteristics for the variable
  - Static- Ordinary memory (globals)
  - Auto - Runtime stack (typical usage)
  - Register - Hardware registers

- Determines when the variable is created and destroyed and how long it will retain its value

- The default storage class for a variable depends on where it is declared

Storage Classes - blocks

```
#include <stdio.h>
{
int main() {
    …….
}
```

- Outside any blocks
  - Storage class is **static**
  - No way to specify any other storage class for these variables
  - Variables are created before the program begins to run and exist throughout its entire execution.
  - They retain whatever value they were assigned until the program completes
  - Static storage

Storage Classes - blocks

```
#include <stdio.h>
int main() {
    …….
}
```

- Within a block
  - **Automatic storage**
  - Stored on the stack
  - Created just before the program execution enters the block in which they are declared;
  - Discarded just as execution leaves that block
  - New copies created each time the block is executed

Storage Classes – static

- Variables declared within a block but with the keyword static changes storage class from automatic to static
- Static storage class exists for the entire duration of the program, rather than just the duration of the block in which it is declared
- NOTE: the changing of the storage class of a variable does not change its scope; it is still accessible by name only from within the block
- FYI: input parameters to a function cannot be declared static, because arguments are always passed on the stack to support recursion

Storage Classes – static (cont)

- When used in function definitions, or declarations of variables that appear outside of blocks
  - The keyword static changes the linkage from external to internal
  - The storage class and scope are not affected
  - Accessible only from within the source file in which they were declared

Storage classes - register

- Can be used on automatic variables to indicate that they should be stored in the machine's hardware registers rather than in memory.
- WHY? To be accessed more efficiently
- FYI – compiler can ignore if necessary i.e. too many designated as register (first come first served) → rest are automatic
- Typically declare heavily used variables as register variables
- Created and destroyed at the same time as automatic variables
- Compiler optimizations will automatically do this

Identifier Storage Class Summary

- Automatic - Stack based
  - Default for local variables
  - Discarded on exit from block
  - Can have auto specifier - Default
  - Can have register specifier
    - Stored in fast registers of the machine if possible instead of RAM

- Static - Useful for globals
  - Default for global variables
    - Declared outside any function blocks
  - Can also be defined within a function w/ keyword static
  - Initialized at compile time and retains its value between call
  - initial value must be a constant

CSE 2421

Linkage

Linkage

- After the individual source files comprising a program are compiled, the object files are linked together with functions from one or more libraries to form the executable program

Linkage Types

- Determines how multiple occurrences of an identifier are treated
- The scope of an identifier is related to its linkage, but the two properties are not the same
- 3 types
  - None - (Inside functions, etc)
    - identifiers that have no linkage are always individuals i.e. multiple declarations of the same identifier are always treated as separate and distinct entities
  - Internal (functions, globals)
    - All declarations of the identifier within one source file refer to a single entity, but declarations of the same identifier in other source files refer to different entities
  - External (extern, header files)
    - All references to an identifier refer to the same entity
    - Global variable known to all functions in the file; declared outside the main() function; automatically initialized to zero.

Exercise

- Write two functions (x and y) that have the desired value scopes (see table below)
- All initializations must be made in the declarations themselves, not by any executable statements in the functions

Name	Type	Storage	Linkage	Scope	Initial Value
a	int	static	external	accessible to x but not y	1
b	char	static	internal	accessible to x and y	2
c	int	automatic	none	local to x	3
d	float	static	none	local to x	4

Exercise - Solution

```
static char b = 2;
void y( void ) {
    //…
}
int a = 1;

void x( void ){
    int c = 3;
    static float d = 4;
}
```

Name	Type	Storage	Linkage	Scope	Initial Value
a	int	static	external	accessible to x but not y	1
b	char	static	internal	accessible to x and y	2
c	int	automatic	none	local to x	3
d	float	static	none	local to x	4